PCCA2 Project Report

Real Root Isolation

Encadrant: Edern Gillot

Hongjing ZHU

Sorbonne Université
Master Informatique M1 – CCA

1 Introduction

The problem of computing and isolating the real roots of a univariate polynomial is a fundamental question in computer algebra. Given a polynomial with real or rational coefficients, one aims to determine all its real roots and to enclose each of them in a disjoint interval with rational endpoints. This process is known as *real root isolation*.

Real root isolation plays an important role in various domains such as symbolic computation, robotics, control theory, and computational geometry. Many applications require not only approximating roots but also guaranteeing their existence and uniqueness within certified intervals.

In this project, we study and implement two classical approaches based on Sturm theory: the *Sturm sequence* and the *Sturm–Habicht sequence*. These methods allow us to count the number of real roots of a polynomial within a given interval by analyzing sign variations.

The main objectives of this project are:

- to implement real root isolation algorithms in C;
- to compare a naive implementation based on Sturm sequences with a more advanced approach using Sturm–Habicht sequences.

2 Theoretical Background

2.1 Notation and Real Root Isolation

Throughout this report, we denote by $\mathbb{Q}$ the field of rational numbers and by $\mathbb{R}$ the field of real numbers.

Let $P(x) \in \mathbb{Q}[x]$ be a univariate polynomial. The problem of *real root isolation* consists in computing a finite set of pairwise disjoint intervals with rational endpoints such that each interval contains exactly one real root of P, and every real root of P belongs to one of these intervals [1].

To isolate the real roots of P, one first needs a method to count the number of real roots in a given interval. This is achieved using Sturm-type methods.

2.2 Sturm Sequences

Let $P \in \mathbb{R}[x]$ be a nonzero polynomial. The Sturm sequence associated with P is the finite sequence $(S_0, S_1, \ldots, S_t)$ defined by:

$$S_0 = P, \qquad S_1 = P',$$

and for every $i \geq 1$,

$$S_{i+1} = -\mathrm{rem}(S_{i-1}, S_i),$$

where $\mathrm{rem}(S_{i-1}, S_i)$ denotes the remainder of the Euclidean division of S_{i-1} by S_i.

The construction stops when a constant polynomial is reached [1].

2.3 Sign Variations

Let $(a_0, a_1, \ldots, a_m)$ be a finite sequence of real numbers. In order to define sign variations, we first remove all zero elements from the sequence and obtain a sequence $(b_0, b_1, \ldots, b_k)$ of nonzero terms.

The number of sign variations of the original sequence, denoted by

$$V(a_0, a_1, \ldots, a_m),$$

is defined as follows:

- if the sequence contains no nonzero element, then $V(a_0, \ldots, a_m) = 0$;
- otherwise, $V(a_0, \ldots, a_m)$ is the number of indices $j \in \{0, \ldots, k-1\}$ such that $b_j \cdot b_{j+1} < 0$.

In other words, a sign variation corresponds to a change of sign between two consecutive nonzero elements. Zero values are ignored when counting sign variations [1].

2.4 Sturm Theorem

Let $a < b$ be two real numbers such that $P(a) \neq 0$ and $P(b) \neq 0$. Let

$$V_P(x) = V\big(S_0(x), S_1(x), \ldots, S_t(x)\big)$$

be the number of sign variations in the evaluations of the Sturm sequence at x.

Then, the number of distinct real roots of P in the interval (a, b) is given by:

$$V_P(a) - V_P(b).$$

This theorem provides an exact method to count real roots in any interval whose endpoints are not roots of the polynomial [1].

2.5 A Root Bound

In order to apply interval subdivision algorithms, one first needs a finite interval that contains all real roots of the polynomial. A classical result is given by the Cauchy bound.

Let

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_0 \quad \text{with } a_n \neq 0.$$

Then every complex root z of P satisfies:

$$|z| \leq 1 + \max_{0 \leq i < n} \left| \frac{a_i}{a_n} \right|.$$

To avoid confusion with the notation $\mathbb{R}$ for the real numbers, we denote this bound by $B(P)$. Therefore, all real roots of P lie in the interval:

$$[-B(P), B(P)].$$

This interval is used as the initial search domain in the implemented algorithms [3].

2.6 Sturm–Habicht Sequences

The Sturm–Habicht sequence is a generalization of the classical Sturm sequence based on subresultant theory. It is constructed from subresultant polynomials together with suitable sign corrections [2].

A key fact for this project is that Sturm–Habicht sequences provide the same root-counting information as Sturm sequences. More precisely, the sign variations of the Sturm–Habicht sequence at the endpoints of an interval allow one to determine the number of real roots in that interval, in the same spirit as Sturm's theorem [2].

This result is the theoretical justification for the second algorithm implemented in this report.

3 Algorithms

3.1 Notation for Algorithmic Construction

For a polynomial $P \in \mathbb{Q}[x]$, we denote:

- $\mathrm{lcoef}(P)$: the leading coefficient of P;
- $\mathrm{rem}(A, B)$: the remainder of the Euclidean division of A by B;
- $\mathrm{Sres}_j(\cdot)$: the j-th subresultant polynomial;
- $\delta_k = (-1)^{k(k+1)/2}$.

3.2 Construction of the Sturm Sequence

Algorithm 1 Construction of the Sturm Sequence

Input: $P \in \mathbb{Q}[x]$, $\deg(P) = p$
Output: $(S_0, \ldots, S_t)$
$S_0 \leftarrow P$
$S_1 \leftarrow P'$
$i \leftarrow 1$
while $i < p$ **do**
  $S_{i+1} \leftarrow -\mathrm{rem}(S_{i-1}, S_i)$
  if $\deg(S_{i+1}) = 0$ **then**
    break
  end if
  $i \leftarrow i + 1$
end while
return $(S_0, \ldots, S_{i+1})$

3.3 Construction of the Sturm–Habicht Sequence

Algorithm 2 Construction of the Sturm–Habicht Sequence

Input: $P \in \mathbb{Q}[x] \setminus \{0\}$, $\deg(P) = p$
Output: $(H_0, \ldots, H_p)$
$L \leftarrow \operatorname{lcoef}(P)$
$D \leftarrow P'$
if $L = 1$ **then**
  $R \leftarrow \operatorname{rem}(D, P)$
  $r \leftarrow \deg(R)$
  $H_p \leftarrow P$
  $H_{p-1} \leftarrow R$
  if $R \neq 0$ **then**
    $H_r \leftarrow \operatorname{lcoef}(R)^{\,p-r+1}\, R$
    for $j \leftarrow r+1$ **to** $p-2$ **do**
      $H_j \leftarrow 0$
    end for
    for $j \leftarrow 0$ **to** $r-1$ **do**
      $H_j \leftarrow \delta_{p-j-1} \operatorname{Sres}_j(P, p, R, r)$
    end for
  else
    for $j \leftarrow 0$ **to** $p-2$ **do**
      $H_j \leftarrow 0$
    end for
  end if
else
  $H_p \leftarrow L\, P$
  $H_{p-1} \leftarrow L\, D$
  for $j \leftarrow 0$ **to** $p-2$ **do**
    $H_j \leftarrow \delta_{p-j-1} \operatorname{Sres}_j(P, p, D, p-1)/L$
  end for
end if
return $(H_0, \ldots, H_p)$

3.4 Root Isolation using Sturm Sequences

Justification. The algorithm relies on Sturm's theorem (Section 2), which ensures that $V_P(a) - V_P(b)$ equals the number of real roots in (a, b) [1]. This value determines whether an interval is discarded, kept, or subdivided. Since the number of real roots is finite and intervals shrink at each step, the algorithm terminates and correctly isolates all roots.

Algorithm 3 Sturm-based Root Isolation

Input: $P \in \mathbb{Q}[x]$
Output: $\mathcal{I}$ (isolating intervals)
Compute $B(P)$
Construct $(S_0, \ldots, S_t)$
$\mathcal{I} \leftarrow \emptyset$
stack $\leftarrow \{[-B(P), B(P)]\}$
while stack not empty **do**
    extract $[a, b]$
    $k \leftarrow V_P(a) - V_P(b)$
    if $k = 0$ **then**
        discard
    else if $k = 1$ **then**
        $\mathcal{I} \leftarrow \mathcal{I} \cup \{[a, b]\}$
    else
        split and push
    end if
end while
return $\mathcal{I}$

3.5 Root Isolation using Sturm–Habicht Sequences

Justification. The Sturm–Habicht sequence provides the same root-counting information as the Sturm sequence (Section 2) [2]. Therefore, the same decision process applies. The finiteness of roots and interval subdivision guarantee termination and correctness.

3.6 Comparison of the Two Approaches

Both methods follow the same recursive isolation strategy. The difference lies in the construction of the sequence used to compute sign variations.

The Sturm sequence relies on Euclidean division and is simpler to implement. The Sturm–Habicht sequence is based on subresultants and determinant computations, making it more complex but better suited to exact arithmetic. In practice, it is expected to behave more efficiently in difficult cases, such as when roots are very close.

Algorithm 4 Sturm–Habicht-based Root Isolation

Input: $P \in \mathbb{Q}[x]$
Output: $\mathcal{I}$
Compute $B(P)$
Construct $(H_0, \ldots, H_p)$
$\mathcal{I} \leftarrow \emptyset$
stack $\leftarrow \{[-B(P), B(P)]\}$
while stack not empty **do**
  extract $[a, b]$
  compute k via sign variations
  if $k = 0$ **then**
    discard
  else if $k = 1$ **then**
    $\mathcal{I} \leftarrow \mathcal{I} \cup \{[a, b]\}$
  else
    split and push
  end if
end while
return $\mathcal{I}$

3.7 Choice of the Subdivision Parameter

In the recursive isolation procedure, an interval containing several roots is subdivided into `nbI` smaller subintervals. Although any fixed subdivision rule ensures correctness, the value of `nbI` has a noticeable impact on performance: a small value leads to deeper recursion, whereas a large value increases the number of endpoint evaluations at each step.

We therefore performed preliminary tests with `nbI` ranging from 2 to 10 on several random instances. In our experiments, the choice

$$\texttt{nbI} = 3$$

gave the best overall compromise. For this reason, all the experiments reported below were carried out with `nbI`=3.

nbI	$(40, [-50, 50])$	$(40, [-50, 50])$	$(40, [-100, 100])$	$(30, [-100, 100])$
2	98.177246	89.687576	132.237703	23.679644
3	57.989370	50.613176	73.630933	14.243418
4	121.293385	88.026356	133.968599	24.658779
5	79.633296	57.497845	84.971422	15.555658
6	146.454914	103.250890	154.838307	27.758197
7	98.457046	69.710708	99.421169	18.130980
8	164.484076	118.062334	175.542970	33.040201
9	109.823818	77.170983	109.985968	21.256938
10	187.272005	139.524913	201.870030	40.214251

Table 1: Preliminary timing results for different values of the subdivision parameter `nbI`.

4 Implementation Details

The source code of the project is available at: `https://github.com/ZHJ0022/sorbonne.info.m1s2.pcca2`

The project is implemented in C and relies on exact rational arithmetic. In order to avoid numerical errors during polynomial manipulations, coefficients are represented with the GMP type `mpq_t`. For some operations involving polynomial evaluation and determinants, the implementation also uses the FLINT library, in particular the types `fmpq_poly_t` and `fmpq_mat_t`.

4.1 General Data Structures

The main data structure is the `Polynomial` structure defined in `include/pcca2.h`. A polynomial is represented by:

- an array `coeffs` of rational coefficients, where `coeffs[i]` stores the coefficient of x^i;
- an integer `degree`;
- an optional array `roots`, used in the test phase to store known roots for verification.

4.2 Memory Management and Auxiliary Functions

The file `src/util.c` contains the auxiliary routines used throughout the project. The function `create_polynomial` allocates a polynomial of a given degree and initializes all coefficients to zero. The function `free_polynomial` releases both the coefficients and the optional stored roots. A deep copy operation is provided by `copy_polynomial`.

The same file also includes `generate_random_polynomial`, which constructs a polynomial from randomly generated rational roots in a prescribed interval. This is useful for testing the correctness of the root isolation algorithms, since

the true roots are known in advance. Finally, the function `afficher_bound` prints the isolated intervals returned by the algorithms.

4.3 Basic Polynomial Operations

The basic algebraic operations are implemented in `src/poly.c`. The derivative is computed by `poly_derivative`. Polynomial multiplication is implemented by the naive `poly_mul_naive`. The Euclidean remainder is computed by `poly_remainder`, which is one of the key ingredients of the naive Sturm sequence.

Several utility operations are also implemented:

- `poly_trim_degree`, which removes trailing zero coefficients and updates the degree;
- `poly_negative`, which multiplies a polynomial by -1;
- `poly_get_lc`, which extracts the leading coefficient;
- `poly_is_zero` and `poly_is_monic`, which test structural properties of a polynomial;
- `poly_scalar_mul` and `poly_scalar_div`, which scale all coefficients by a rational factor.

Polynomial evaluation is implemented in two ways. The function `poly_calculate_sign` uses Horner's method directly on GMP rationals. In addition, the pair `polynomialConv` and `poly_sign_flint` converts the polynomial to FLINT format and evaluates it using FLINT routines. In the recursive isolation step, the implementation uses the FLINT-based evaluation in order to compute sign variations efficiently.

4.4 Root Bounds and Sign Variations

The file `src/calculate.c` gathers the elementary numerical routines needed by both isolation methods.

The function `cauchy_bound` computes the classical Cauchy bound

$$R = 1 + \max_{0 \le i < n} \left| \frac{a_i}{a_n} \right|,$$

which guarantees that all real roots lie in the interval $[-R, R]$. This interval is used as the starting point for the recursive subdivision procedure.

The function `nb_sign_change` computes the number of sign changes in a sequence while ignoring zeros, which is exactly the quantity needed in Sturm's theorem. The file also contains `verify_interval`, a testing function that checks whether every known root of a generated polynomial belongs to one of the isolated intervals. Finally, the helper `delta_sign` computes the factor $(-1)^{k(k+1)/2}$ used in the construction of the Sturm–Habicht sequence.

4.5 Naive Sturm Sequence Implementation

The naive implementation of Sturm's method is contained in `src/sturmNaif.c`. The function `sturm_naif` first constructs the Sturm sequence:

$$S_0 = P, \qquad S_1 = P', \qquad S_{i+1} = -\mathrm{rem}(S_{i-1}, S_i).$$

The construction stops when a constant polynomial is reached.

Once the sequence has been built, the algorithm computes a global search interval using the Cauchy bound and calls the recursive function `bound_recu`. This function evaluates the whole Sturm sequence at the endpoints of a subinterval, computes the number of sign variations at both ends, and deduces the number of roots in the interval from their difference.

The recursive strategy is the following:

- if an interval contains no root, it is discarded;
- if it contains exactly one root, it is returned as an isolating interval;
- otherwise, the interval is subdivided into smaller subintervals and the process is applied recursively.

The output format is an array of rational values: the first entry stores the number of isolated real roots, and the remaining entries store the interval endpoints.

4.6 Subresultants and Sylvester Matrices

The file `src/subresultant.c` implements the subresultant machinery needed for the Sturm–Habicht approach. First, the code allocates rational matrices with `alloc_matrix`. The function `create_sylv_j_matrix` constructs the truncated Sylvester matrix associated with two polynomials P and S and an index j.

The determinant computations are performed by FLINT. Since the rest of the project uses GMP rationals, the code provides two conversion functions, `mpq_to_fmpq` and `fmpq_to_mpq`, to move between the two libraries. The function `sylv_j_det` extracts the appropriate square minor and computes its determinant. From these determinants, the function `subresultant_j` reconstructs the j-th subresultant polynomial.

This part of the project is more algebraically involved than the naive Sturm implementation, since it explicitly manipulates Sylvester matrices and determinant-based constructions.

4.7 Sturm–Habicht Sequence Implementation

The file `src/sturmHabicht.c` implements the second root isolation method. The function `create_sturm_habicht` constructs the full Sturm–Habicht sequence of a polynomial. The implementation distinguishes two main cases:

- the case where the polynomial is monic, where the sequence starts from P and the remainder of the division of P' by P;
- the case where the leading coefficient is not equal to 1, where additional scaling by the leading coefficient is required.

The sequence is completed using subresultants together with the sign factor $(-1)^{k(k+1)/2}$. Once the sequence has been constructed, the function `sturm_habicht` applies the same recursive isolation routine `bound_recu` as in the naive Sturm method.

5 Experimental Protocol and Validation

5.1 Test Instances

In order to validate the implementation, we consider polynomials generated by the function `generate_random_polynomial`. This function constructs a polynomial from randomly generated rational roots within a prescribed interval.

This approach provides a convenient testing framework, since the exact real roots of the polynomial are known in advance. As a result, it becomes possible to directly compare the computed isolating intervals with the true roots.

Each test instance is processed independently by both the naive Sturm method or the Sturm–Habicht method.

5.2 Validation of Isolating Intervals

The correctness of the computed isolating intervals is verified using the function `verify_interval`.

Given a polynomial with known roots, and a list of intervals returned by the isolation algorithm, the validation proceeds as follows:

- for each known root, check whether it belongs to at least one of the computed intervals;
- ensure that each interval contains a root;
- verify that the intervals are disjoint.

This verification step guarantees that the output satisfies the definition of real root isolation: every real root is contained in an interval, and each interval isolates exactly one root.

5.3 Evaluation Criteria

The implemented methods are evaluated according to two main criteria.

Correctness. The primary objective is to ensure that both algorithms correctly isolate all real roots. This is assessed through the validation procedure described above, by comparing the computed intervals with the known roots of the test polynomials.

Execution Time. The computational efficiency of the algorithms is also considered. We compare the relative execution time of the naive Sturm method and the Sturm–Habicht method on the same test instances. This comparison provides insight into the practical cost of the more advanced algebraic constructions used in the Sturm–Habicht approach.

6 Tests and Results

6.1 Structured Polynomials

As a first family of experiments, we considered the structured polynomials

$$P_n(x) = \prod_{k=1}^{n} (x - k),$$

whose real roots are the integers $1, \ldots, n$. These tests were implemented in `test1.c`. For each value of n, we measured:

- the time needed to construct the Sturm sequence;
- the time needed to compute the isolating intervals with the naive Sturm method;
- the time needed to construct the Sturm–Habicht sequence;
- the time needed to compute the isolating intervals with the Sturm–Habicht method.

All tested instances were correctly solved by both methods.

Polynomial	Sturm naïf: suite (s)	Sturm naïf: intervals (s)	Sturm–Habicht: suite (s)	Sturm–Habicht: intervals (s)
$(x-1)\cdots(x-5)$	0.000044	0.000638	0.000551	0.000146
$(x-1)\cdots(x-10)$	0.000049	0.001940	0.000393	0.001599
$(x-1)\cdots(x-15)$	0.000068	0.009779	0.004573	0.009929
$(x-1)\cdots(x-20)$	0.000120	0.037683	0.036871	0.036150
$(x-1)\cdots(x-25)$	0.000174	0.102967	0.182652	0.102565
$(x-1)\cdots(x-30)$	0.000311	0.273682	0.588022	0.273682

Table 2: Execution times for structured polynomials $P_n(x) = \prod_{k=1}^{n}(x-k)$.

Table 2 shows that both methods correctly isolate the real roots on all tested examples. The naive Sturm method is systematically faster for the construction

of the sequence, while the interval computation times are very close for the two methods.

This behavior is consistent with the theoretical structure of the algorithms. The naive Sturm sequence is obtained by repeated Euclidean remainders, whereas the Sturm–Habicht sequence requires subresultant computations, which are algebraically more expensive. On the other hand, once the sequence has been constructed, both methods use the same recursive subdivision procedure to isolate the roots, which explains why the interval computation times are of the same order.

For this family of examples, the roots are well separated and regularly distributed. As a consequence, the isolation step is not especially difficult, and the extra cost of constructing the Sturm–Habicht sequence is not compensated by a significant gain during subdivision. In this regime, the naive Sturm method is therefore more efficient overall.

6.2 Random Polynomials

In order to evaluate the two methods on less structured inputs, we generated random polynomials of degree 40 whose roots lie in the interval $[-100, 100]$. These experiments were implemented in `test3.c`. For each random instance, we measured:

- the time needed to construct the Sturm sequence;
- the time needed to compute the isolating intervals with the naive Sturm method;
- the time needed to construct the Sturm–Habicht sequence;
- the time needed to compute the isolating intervals with the Sturm–Habicht method.

We repeated the experiment on five independently generated random instances, using the same subdivision parameter `nbI = 3`. In all cases, both methods returned correct isolating intervals.

Test	Sturm naïf: suite (s)	Sturm naïf: intervals (s)	Sturm–Habicht: suite (s)	Sturm–Habicht: intervals (s)
Random #1	1.289474	64.596542	7.084340	3.248623
Random #2	1.331726	68.193595	7.717345	3.166335
Random #3	1.359677	87.484930	9.046607	3.960729
Random #4	1.316596	88.534319	8.871142	4.085914
Random #5	1.315487	78.868288	7.819456	3.273130

Table 3: Execution times for five random polynomials of degree 40 with roots in $[-100, 100]$.

The results in Table 3 show a clear contrast between the two phases of the computation. As already observed on structured examples, the naive Sturm

method is faster for the construction of the sequence itself. However, the gain obtained by Sturm–Habicht during interval computation is much larger than the loss incurred during sequence construction.

More precisely, for all five random instances, the time required to compute the isolating intervals with Sturm–Habicht is around 3 to 4 seconds, whereas the corresponding time for the naive Sturm method ranges from about 65 to 89 seconds. Thus, even though the Sturm–Habicht sequence is more expensive to construct, the total running time is significantly smaller for the Sturm–Habicht method.

This behavior is consistent with the theoretical discussion. The construction of the Sturm–Habicht sequence is algebraically more involved, since it is based on subresultants rather than direct Euclidean remainders. On the other hand, once the sequence is available, the evaluation phase becomes much more efficient, which is particularly important in root isolation, where many interval endpoints must be tested recursively.

To further illustrate this phenomenon, we also tested a larger random instance in `test4.c`, corresponding to a polynomial of degree 80 with roots in $[-1000, 1000]$. In this case, the naive Sturm method did not finish within one hour and was stopped manually, while the Sturm–Habicht method completed the computation in 1901.705380 seconds (approximately 31.7 minutes). This large-scale example confirms that, for harder random instances, the practical advantage of the Sturm–Habicht approach becomes even more significant.

Polynomial	Sturm naïf (total time)	Sturm–Habicht (total time)
Random degree 80, roots in $[-1000, 1000]$	> 1 hour	1901.705380 s

Table 4: Large random instance illustrating the scalability gap between the two methods.

This additional experiment is not meant to provide a statistical average, but rather to highlight the asymptotic tendency already visible in Table 3: for sufficiently large random inputs, the naive Sturm method becomes impractical, while the Sturm–Habicht method remains feasible.

6.3 Close-Root Polynomials

In order to complement the structured and random test families, we also considered polynomials whose real roots are deliberately chosen to be very close to each other. The purpose of this experiment is to construct instances that are, at least in principle, more challenging for root isolation, since very small separations between consecutive roots may force the subdivision process to examine narrower intervals before each root is isolated.

More precisely, for fixed integers $d \geq 1$ and $k \geq 1$, we considered the family

$$P_{d,k}(x) = \prod_{i=0}^{d-1} (2^k x - i),$$

whose roots are

$$\left\{ \frac{i}{2^k} \;\middle|\; 0 \le i \le d-1 \right\}.$$

Hence, the distance between two consecutive roots is exactly 2^{-k}, so increasing k produces increasingly clustered roots. In the experiments below, we fixed $d = 40$ and tested the values $k = 16, 24, 32$.

Integer-coefficient version. The first tests were performed on the polynomial $P_{d,k}(x)$ itself, which has integer coefficients. The results are reported in Table 5.

Polynomial	Sturm naïf: suite (s)	Sturm naïf: intervals (s)	Sturm–Habicht: suite (s)	Sturm–Habicht: intervals (s)
$\prod_{i=0}^{39}(2^{16}x - i)$	0.000751	0.059766	23.335066	0.299223
$\prod_{i=0}^{39}(2^{24}x - i)$	0.000721	0.086052	37.422677	0.591533
$\prod_{i=0}^{39}(2^{32}x - i)$	0.000789	0.136288	54.282755	0.926066

Table 5: Execution times for close-root polynomials in integer-coefficient form.

All instances were solved correctly by both methods. However, the observed behaviour differs from the initial expectation. One might expect close roots to favour the Sturm–Habicht approach during the interval-isolation phase, because many endpoint evaluations may be required when roots are difficult to separate. In our implementation, this effect is not the dominant one.

Indeed, the most striking feature of Table 5 is the very large cost of constructing the Sturm–Habicht sequence, which increases dramatically with k, from about 23 seconds to more than 54 seconds, while the construction time of the naive Sturm sequence remains negligible. This indicates that, for this family, the main difficulty does not come from root proximity alone, but rather from coefficient growth. Although the roots remain in a very small interval near 0, the integer-coefficient form $\prod_{i=0}^{d-1}(2^k x - i)$ produces coefficients whose size increases rapidly with k. Since our implementation of the Sturm–Habicht sequence relies on subresultants and determinant computations, it is particularly sensitive to this coefficient growth.

On the other hand, the interval-computation phase remains comparatively cheap for both methods. Even though the roots are close, they are all concentrated in a very small region, so the recursive subdivision remains localized. In particular, many evaluations far from the root cluster are avoided, and the total number of expensive endpoint evaluations appears to remain limited. This is consistent with the fact that the interval times in Table 5 remain much smaller than those observed on the random polynomials of Section 5.2. Consequently, the potential advantage of Sturm–Habicht in the evaluation phase is not sufficient here to compensate for the cost of constructing the sequence, and the total running time remains larger than for the naive Sturm method.

Monic version. To distinguish the effect of root proximity from that of coefficient growth, we then performed the same experiment on the monic polynomial

$$\widetilde{P}_{d,k}(x) = \prod_{i=0}^{d-1} \left(x - \frac{i}{2^k} \right),$$

which has exactly the same roots as $P_{d,k}(x)$, but a much smaller leading coefficient and, more generally, a much milder coefficient growth. The corresponding results are given in Table 6.

Polynomial	Sturm naïf: suite (s)	Sturm naïf: intervals (s)	Sturm–Habicht: suite (s)	Sturm–Habicht: intervals (s)
$\prod_{i=0}^{39} \left(x - \frac{i}{2^{16}}\right)$	0.000028	0.000383	0.085064	0.000015
$\prod_{i=0}^{39} \left(x - \frac{i}{2^{24}}\right)$	0.000015	0.000007	0.086100	0.000022
$\prod_{i=0}^{39} \left(x - \frac{i}{2^{32}}\right)$	0.000041	0.000009	0.088969	0.000016

Table 6: Execution times for close-root polynomials in monic form.

Here again, all instances were solved correctly by both methods. The comparison with Table 5 is particularly instructive. Once the coefficient growth is removed, both methods become extremely fast, and the construction time of the Sturm–Habicht sequence drops by several orders of magnitude. This confirms that the very large running times observed in the integer-coefficient version were mainly caused by the arithmetic complexity of the coefficients, rather than by root proximity itself.

Moreover, the interval-computation times in Table 6 are extremely small for both methods. This suggests that, in our implementation, these close-root instances do not generate a sufficiently large subdivision tree for the isolation phase to become expensive. In particular, although the roots are very close to each other, they all lie in a very small interval, so the recursive search remains highly localized. For this reason, the difficulty of separating nearby roots does not translate here into a large practical cost.

Conclusion. These experiments show that close-root instances must be interpreted with care. The family $\prod_{i=0}^{d-1}(2^k x - i)$ does not isolate the effect of root proximity alone, because increasing k simultaneously decreases the root separation and increases the size of the coefficients. In our implementation, the second effect is dominant for the Sturm–Habicht method. By contrast, the monic family

$$\prod_{i=0}^{d-1} \left(x - \frac{i}{2^k} \right)$$

shows that, when coefficient growth is controlled, close roots by themselves do not necessarily make the interval-isolation phase expensive in practice, at least for the range of parameters considered here.

7 Bibliography

1. Saugata Basu, Richard Pollack, and Marie-Françoise Roy. 2006. *Algorithms in Real Algebraic Geometry*, 2nd ed. Springer.

2. Laureano Gonzalez, Henri Lombardi, Tomás Recio, and Marie-Françoise Roy. 1989. *Sturm-Habicht Sequence*. In Proceedings of the International Symposium on Symbolic and Algebraic Computation (ISSAC).

3. Wikipedia contributors. *Geometrical Properties of Polynomial Roots* `https://en.wikipedia.org/wiki/Geometrical_properties_of_polynomial_roots`